

Lab 6.3. Fragments and the Navigation Component

1. Before you begin
2. Starter Code
3. Fragments and the fragment lifecycle
4. Create Fragment and layout Files
5. Implement LetterListFragment
6. Convert DetailActivity to WordListFragment
7. Jetpack Navigation Component
8. Using the Navigation Graph
9. Getting Arguments in WordListFragment
10. Update Fragment Labels
11. Solution code
12. Summary
13. Learn more

[1. Before you begin](#)

In the Activities and Intents codelab, you added intents in the [Words](#) app, to navigate between two activities. While this is a useful navigation pattern to know, it's only part of the story of making dynamic user interfaces for your apps. Many Android apps don't need a separate activity for every screen. In fact, many common UI patterns, such as tabs, exist within a single activity, using something called *fragments*.



Search contacts



No one is on your speed dial yet

[ADD A FAVORITE](#)



Favorites



Recents



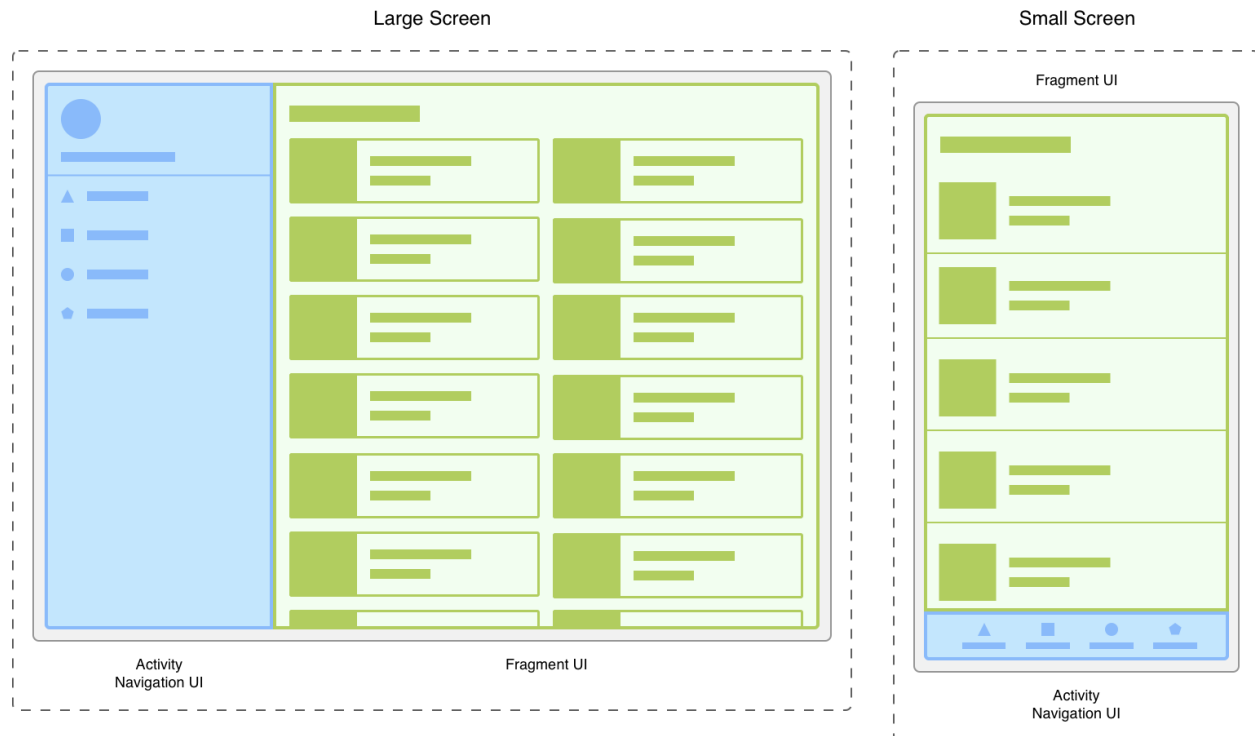
Contacts



Voicemail

A [fragment](#) is a reusable piece of UI; fragments can be reused and embedded in one or more activities. In the above screenshot, tapping on a tab doesn't trigger an intent to display the next screen. Instead, switching tabs simply swaps out the previous fragment with another fragment. All of this happens without launching another activity.

You can even show multiple fragments at once on a single screen, such as a master-detail layout for tablet devices. In the example below, both the navigation UI on the left and the content on the right can each be contained in a separate fragment. Both fragments exist simultaneously in the same activity.



As you can see, fragments are an integral part of building high quality apps. In this codelab, you'll learn the basics of fragments, and convert the Words app to use them. You'll also learn how to use the Jetpack [Navigation component](#) and work with a new resource file called the **Navigation Graph** to navigate between fragments in the same host activity. By the end of this codelab, you'll come away with the foundational skills for implementing fragments in your next app.

Prerequisites

Before completing this codelab, you should know

- How to add resource XML files and Kotlin files to an Android Studio project.
- How the activity lifecycle works at a high level.
- How to override and implement methods in an existing class.
- How to create instances of Kotlin classes, access class properties, and call methods.

- Basic familiarity with nullable and non-nullable values and know how to safely handle null values.

What you'll learn

- How the fragment lifecycle differs from the activity lifecycle.
- How to convert an existing activity into a fragment.
- How to add destinations to a navigation graph, and pass data between fragments while using the Safe Args plugin.

What you'll build

- You'll modify the Words app to use a single activity and multiple fragments, and navigate between fragments with the Navigation Component.

What you need

- A computer with Android Studio installed.
- Solution code of Words app from the Activities and Intents codelab

2. Starter Code

In this codelab, you'll pick up where you left off with the Words app at the end of the Activities and Intents codelab. If you've already completed the codelab for activities and intents, feel free to use your code as a starting point. You can alternately download the code up until this point from GitHub.

Download the starter code for this codelab

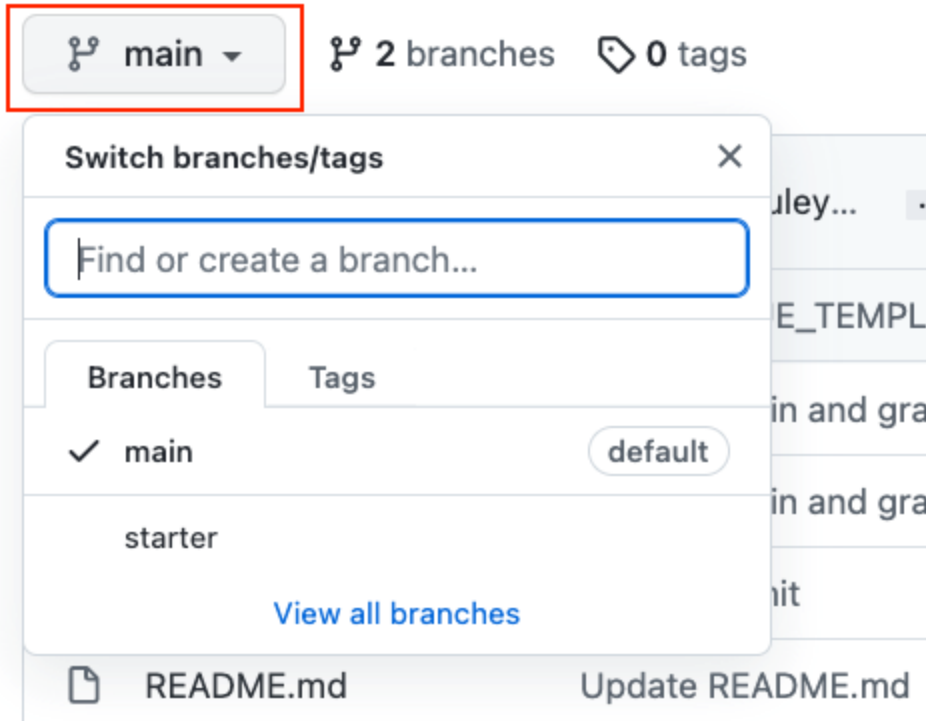
This codelab provides starter code for you to extend with features taught in this codelab. Starter code may contain code that is familiar to you from previous codelabs. It may also contain code that is unfamiliar to you, and that you will learn about in later codelabs.

If you use the starter code from GitHub, note that the folder name is `android-basics-kotlin-words-app-activities`. Select this folder when you open the project in Android Studio.

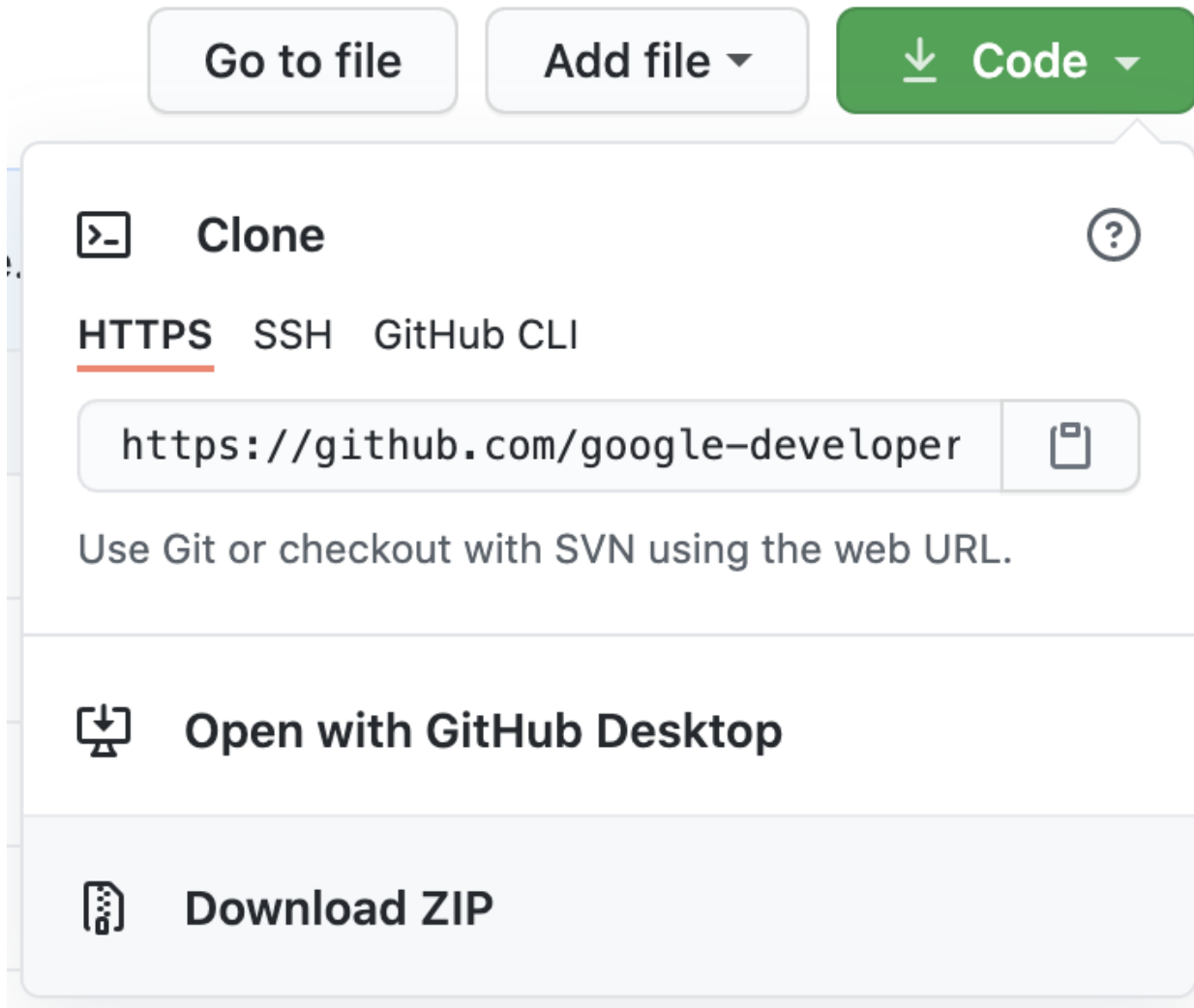
Starter Code URL: <https://github.com/google-developer-training/android-basics-kotlin-words-app>

Module name with starter code: `activities`

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



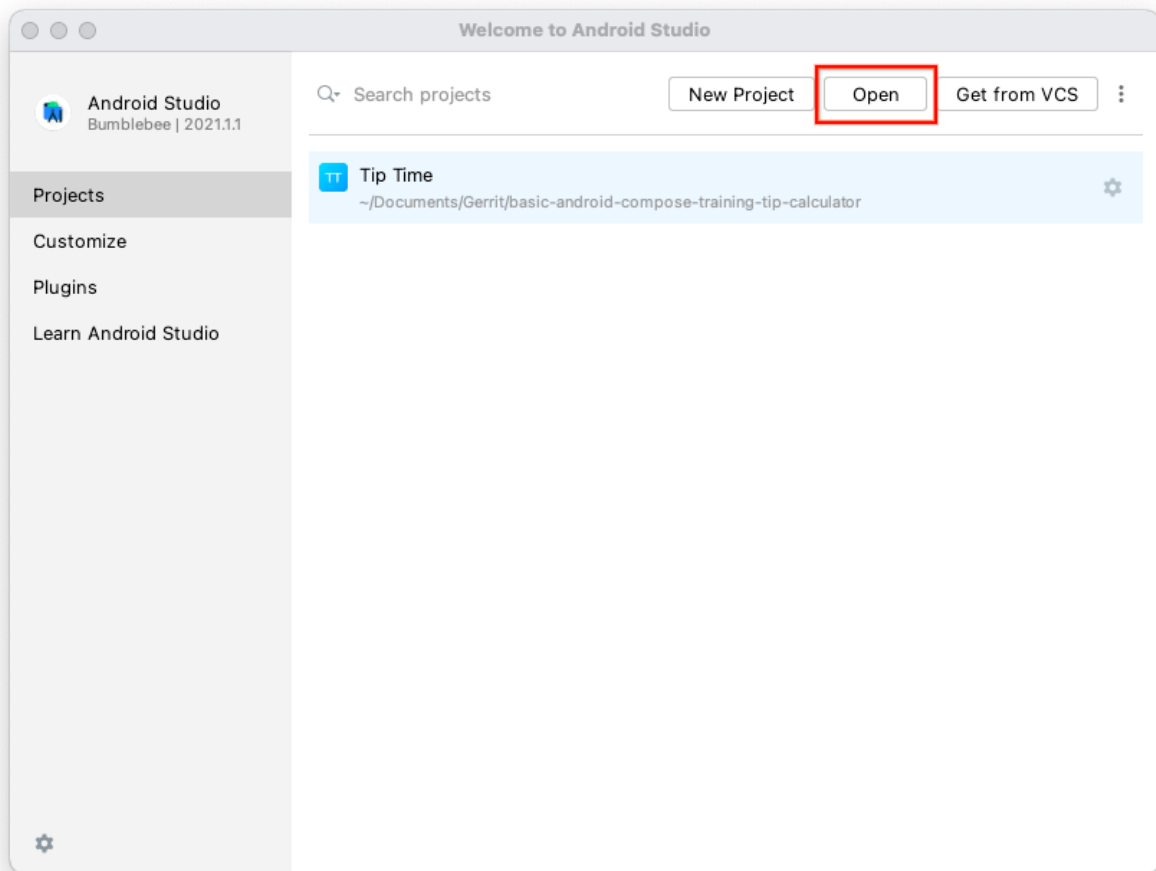
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



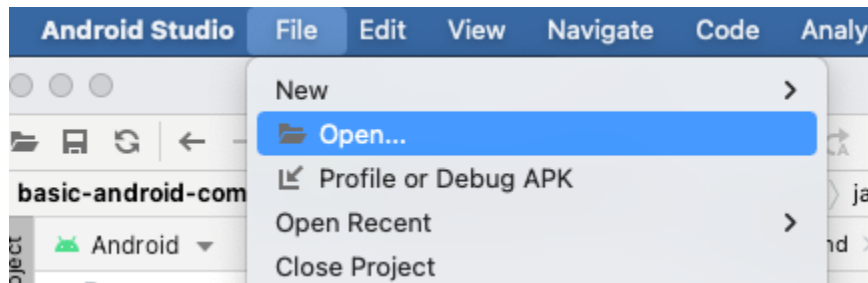
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

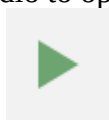
1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

3. Fragments and the fragment lifecycle

A fragment is simply a reusable piece of your app's user interface. Like activities, fragments have a lifecycle and can respond to user input. A fragment is always contained within the view hierarchy of an activity when it is shown onscreen. Due to their emphasis on reusability and modularity, it's even possible for multiple fragments to be hosted simultaneously by a single activity. Each fragment manages its own separate lifecycle.

Fragment lifecycle

Like activities, fragments can be initialized and removed from memory, and throughout their existence, appear, disappear, and reappear onscreen. Also, just like activities, fragments have a lifecycle with several states, and provide several methods you can override to respond to transitions between them. The fragment lifecycle has five states, represented by the [*Lifecycle.State*](#) enum.

- **INITIALIZED:** A new instance of the fragment has been instantiated.
- **CREATED:** The first fragment lifecycle methods are called. During this state, the view associated with the fragment is also created.
- **STARTED:** The fragment is visible onscreen but does not have "focus", meaning it can't respond to user input.
- **RESUMED:** The fragment is visible and has focus.
- **DESTROYED:** The fragment object has been de-instantiated.

Also similar to activities, the [*Fragment*](#) class provides many methods that you can override to respond to lifecycle events.

- `onCreate()`: The fragment has been instantiated and is in the **CREATED** state. However, its corresponding view has not been created yet.
- `onCreateView()`: This method is where you inflate the layout. The fragment has entered the **CREATED** state.
- `onViewCreated()`: This is called after the view is created. In this method, you would typically bind specific views to properties by calling `findViewById()`.
- `onStart()`: The fragment has entered the **STARTED** state.
- `onResume()`: The fragment has entered the **RESUMED** state and now has focus (can respond to user input).
- `onPause()`: The fragment has re-entered the **STARTED** state. The UI is visible to the user.
- `onStop()`: The fragment has re-entered the **CREATED** state. The object is instantiated but is no longer presented on screen.
- `onDestroyView()`: Called right before the fragment enters the **DESTROYED** state. The view has already been removed from memory, but the fragment object still exists.
- `onDestroy()`: The fragment enters the **DESTROYED** state.

The chart below summarizes the fragment lifecycle, and the transitions between states.

Lifecycle State	Callback
CREATED	onCreate()
	onCreateView()
	onViewCreated()
STARTED	onStart()
RESUMED	onResume()
STARTED	onPause()
CREATED	onStop()
	onDestroyView()
DESTROYED	onDestroy()

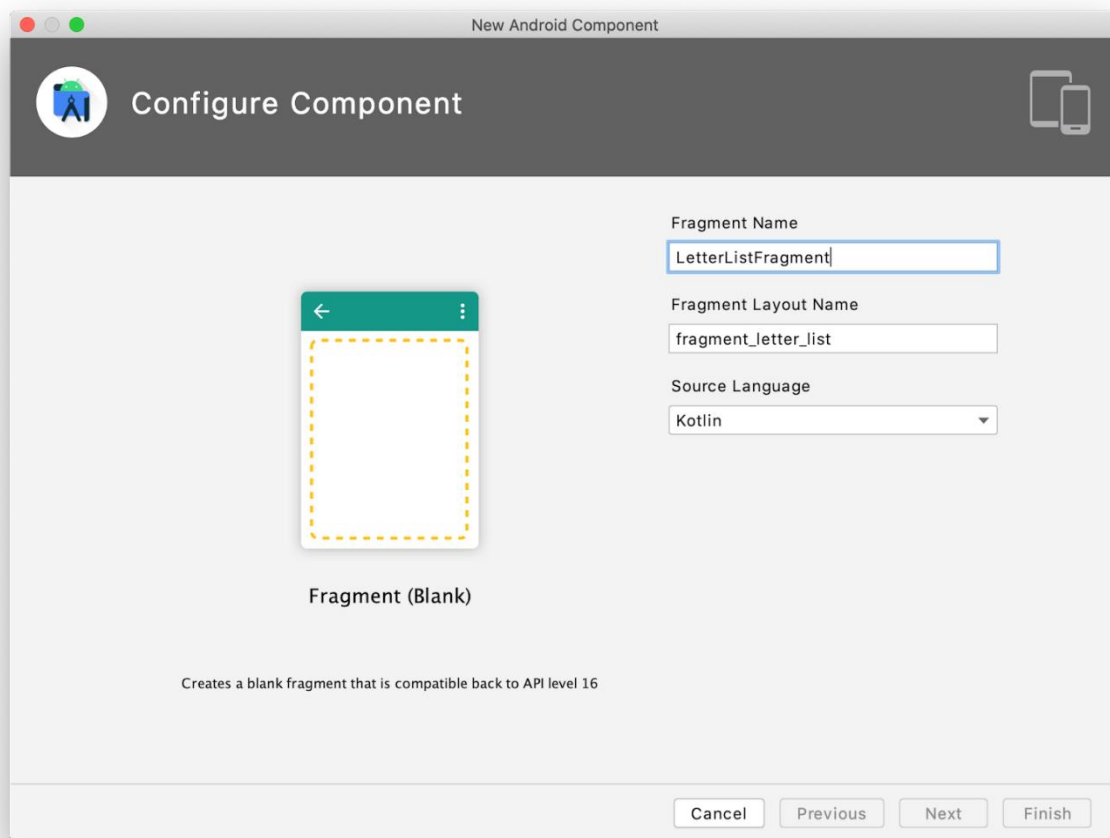
The lifecycle states and callback methods are quite similar to those used for activities. However, keep in mind the difference with the `onCreate()` method. With activities, you would use this method to inflate the layout and bind views. However, in the fragment lifecycle, `onCreate()` is called before the view is created, so you can't inflate the layout here. Instead, you do this in `onCreateView()`. Then, after the view has been created, the `onViewCreated()` method is called, where you can then bind properties to specific views.

While that probably sounded like a lot of theory, you now know the basics of how fragments work, and how they're similar and different to activities. For the remainder of this codelab, you'll put that knowledge to work. First, you'll migrate the Words app you worked on previously to use a fragment based layout. Then, you'll implement navigation between fragments within a single activity.

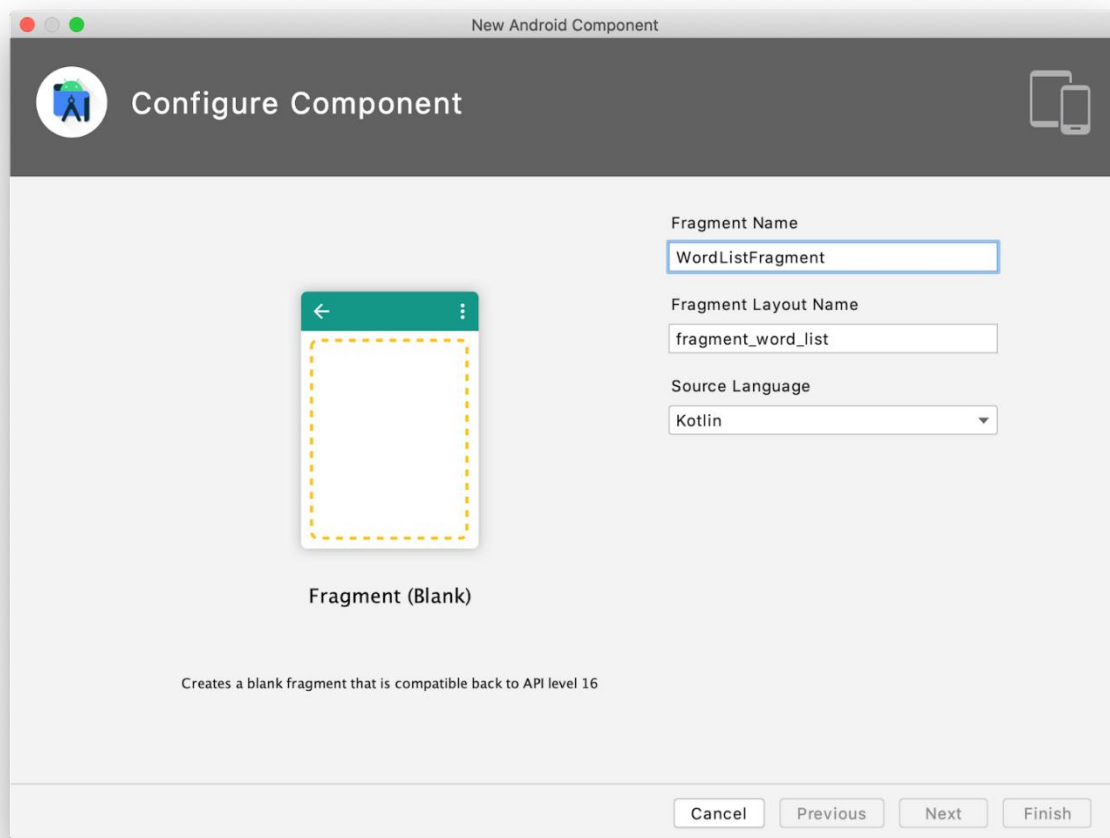
4. Create Fragment and layout Files

As with activities, each fragment you add will consist of two files—an XML file for the layout and a Kotlin class to display data and handle user interactions. You'll add a fragment for both the letter list and the word list.

1. With **app** selected in the Project Navigator, add the following fragments (**File > New > Fragment > Fragment (Blank)**) and both a class and layout file should be generated for each.
 - For the first fragment, set the **Fragment Name** to `LetterListFragment`. The **Fragment Layout Name** should populate as `fragment_letter_list`.



- For the second fragment, set the **Fragment Name** to `WordListFragment`. The **Fragment Layout Name** should populate as `fragment_word_list.xml`.



2. The generated Kotlin classes for both fragments contain a lot of boilerplate code commonly used when implementing fragments. However, as you're learning about fragments for the first time, go ahead and delete everything except the class declaration for `LetterListFragment` and `WordListFragment` from both files. We'll walk you through implementing the fragments from scratch so that you know how all of the code works. After deleting the boilerplate code, the Kotlin files should look as follows.

LetterListFragment.kt

```
package com.example.wordsapp

import androidx.fragment.app.Fragment

class LetterListFragment : Fragment() {

}
```

WordListFragment.kt

```
package com.example.wordsapp
```

```
import androidx.fragment.app.Fragment

class WordListFragment : Fragment() {

}
```

3. Copy the contents of `activity_main.xml` into `fragment_letter_list.xml` and the contents of `activity_detail.xml` into `fragment_word_list.xml`. Update `tools:context` in `fragment_letter_list.xml` to `.LetterListFragment` and `tools:context` in `fragment_word_list.xml` to `.WordListFragment`.

After the changes, the fragment layout files should look as follows.

fragment_letter_list.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".LetterListFragment">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:clipToPadding="false"
        android:padding="16dp" />

</FrameLayout>
```

fragment_word_list.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".WordListFragment">

    <androidx.recyclerview.widget.RecyclerView
        android:id="@+id/recycler_view"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:clipToPadding="false"
        android:padding="16dp"
        tools:listitem="@layout/item_view" />

</FrameLayout>
```

5. Implement LetterListFragment

As with activities, you need to inflate the layout and bind individual views. There are just a few minor differences when working with the fragment lifecycle. We'll walk you through the process for setting up the `LetterListFragment`, and then you'll get the chance to do the same for `WordListFragment`.

To implement view binding in `LetterListFragment`, you first need to get a nullable reference to `FragmentLetterListBinding`. Binding classes like this are generated by Android Studio for each layout file, when the `viewBinding` property is enabled under the `buildFeatures` section of the **build.gradle** file. You just need to assign properties in your fragment class for each view in the `FragmentLetterListBinding`.

The type should be `FragmentLetterListBinding?` and it should have an initial value of `null`. Why make it nullable? Because you can't inflate the layout until `onCreateView()` is called. There's a period of time in-between when the instance of `LetterListFragment` is created (when its lifecycle begins with `onCreate()`) and when this property is actually usable. Also keep in mind that fragments' views can be created and destroyed several times throughout the fragment's lifecycle. For this reason you also need to reset the value in another lifecycle method, `onDestroyView()`.

1. In `LetterListFragment.kt`, start by getting a reference to the `FragmentLetterListBinding`, and name the reference `_binding`.

```
private var _binding: FragmentLetterListBinding? = null
```

Because it's nullable, every time you access a property of `_binding`, (e.g. `_binding?.someView`) you need to include the `?` for null safety. However, that doesn't mean you have to litter your code with question marks just because of one null value. If you're certain a value won't be null when you access it, you can append `!!` to its type name. Then you can access it like any other property, without the `?` operator.

NOTE: When making a variable nullable using `!!`, it's a good idea to limit its usage to only one or a few places where you know the value won't be null, just like you know `_binding` will have a value after it is assigned in `onCreateView()`. Accessing a nullable value in this manner is dangerous and can lead to crashes, so use sparingly, if at all.

2. Create a new property, called `binding` (without the underscore) and set it equal to `_binding!!`.

```
private val binding get() = _binding!!
```

Here, `get()` means this property is "get-only". That means you can **get** the value, but once assigned (as it is here), you can't assign it to something else.

NOTE: In Kotlin, and programming in general, you'll often encounter property names preceded by an underscore. This typically means that the property isn't intended to be accessed directly. In your case, you access the view binding in `LetterListFragment` with the binding property. However, the `_binding` property does not need to be accessed outside of `LetterListFragment`.

3. To display the options menu, override `onCreate()`. Inside `onCreate()` call `setHasOptionsMenu()` passing in `true`.

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)
    setHasOptionsMenu(true)
}
```

4. Remember that with fragments, the layout is inflated in `onCreateView()`. Implement `onCreateView()` by inflating the view, setting the value of `_binding`, and returning the root view.

```
override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {
    _binding = FragmentLetterListBinding.inflate(inflater, container, false)
    val view = binding.root
    return view
}
```

5. Below the `binding` property, create a property for the `recyclerView`.

```
private lateinit var recyclerView: RecyclerView
```

6. Then set the value of the `recyclerView` property in `onViewCreated()`, and call `chooseLayout()` like you did in `MainActivity`. You'll move the `chooseLayout()` method into `LetterListFragment` soon, so don't worry that there's an error.

```
override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    recyclerView = binding.recyclerView
    chooseLayout()
}
```

Notice how the binding class already created a property for `recyclerView`, and you don't need to call `findViewById()` for each view.

7. Finally, in `onDestroyView()`, reset the `_binding` property to `null`, as the view no longer exists.

```
override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}
```

8. The only other thing to note is there are some subtle differences with the `onCreateOptionsMenu()` method when working with fragments. While the `Activity` class has a global property called `menuInflater`, `Fragment` does not have this property. The menu inflater is instead passed into `onCreateOptionsMenu()`. Also note that the `onCreateOptionsMenu()` method used with fragments doesn't require a return statement. Implement the method as shown:

```
override fun onCreateOptionsMenu(menu: Menu, inflater: MenuInflater) {
    inflater.inflate(R.menu.layout_menu, menu)

    val layoutButton = menu.findItem(R.id.action_switch_layout)
    setIcon(layoutButton)
}
```

9. Move the remaining code for `chooseLayout()`, `setIcon()`, and `onOptionsItemSelected()` from `MainActivity` as-is. The only other difference to note is that, unlike an activity, a fragment is not a [Context](#). You can't pass in this (referring to the fragment object) as the layout manager's context. However, fragments provide a `context` property you can use instead. The rest of the code is identical to `MainActivity`.

```
private fun chooseLayout() {
    when (isLinearLayoutManager) {
        true -> {
            recyclerView.layoutManager = LinearLayoutManager(context)
            recyclerView.adapter = LetterAdapter()
        }
        false -> {
            recyclerView.layoutManager = GridLayoutManager(context, 4)
            recyclerView.adapter = LetterAdapter()
        }
    }
}

private fun setIcon(menuItem: MenuItem?) {
    if (menuItem == null)
        return

    menuItem.icon =
        if (isLinearLayoutManager)
            ContextCompat.getDrawable(this.requireContext(),
R.drawable.ic_grid_layout)
        else ContextCompat.getDrawable(this.requireContext(),
R.drawable.ic_linear_layout)
}

override fun onOptionsItemSelected(item: MenuItem): Boolean {
    return when (item.itemId) {
        R.id.action_switch_layout -> {
            isLinearLayoutManager = !isLinearLayoutManager
            chooseLayout()
            setIcon(item)

            return true
        }
    }
}
```

```

        }

        else -> super.onOptionsItemSelected(item)
    }
}

```

Note: [`requireContext\(\)`](#) returns the `Context` this fragment is currently associated with.

10. Finally, copy over the `isLinearLayoutManager` property from `MainActivity`. Put this right below the declaration of the `recyclerView` property.

```
private var isLinearLayoutManager = true
```

11. Now that all the functionality has been moved to `LetterListFragment`, all the `MainActivity` class needs to do is inflate the layout so that the fragment is displayed in the view. Go ahead and delete everything except `onCreate()` from `MainActivity`. After the changes, `MainActivity` should contain only the following.

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    val binding = ActivityMainBinding.inflate(layoutInflater)
    setContentView(binding.root)
}

```

Your turn

That's it for migrating `MainActivity` to `LettersListFragment`. Migrating the `DetailActivity` is almost identical. Perform the following steps to migrate the code to `WordListFragment`.

1. Copy the companion object from `DetailActivity` to `WordListFragment`. Make sure the reference to `SEARCH_PREFIX` in `WordAdapter` is updated to reference `WordListFragment`.
2. Add a `_binding` variable. The variable should be nullable and have `null` as its initial value.
3. Add a get-only variable called `binding` equal to the `_binding` variable.
4. Inflate the layout in `onCreateView()`, setting the value of `_binding` and returning the root view.
5. Perform any remaining setup in `onViewCreated()`: get a reference to the recycler view, set its layout manager and adapter, and add its item decoration. You'll need to get the letter from the intent. As fragments don't have an `intent` property and shouldn't normally access the intent of the parent activity. For now, you refer to `activity.intent` (rather than `intent` in `DetailActivity`) to get the extras.
6. Reset `_binding` to null in `onDestroyView`.
7. Delete the remaining code from `DetailActivity`, leaving only the `onCreate()` method.

Try to go through the steps on your own before moving on. A detailed walkthrough is available on the next step.

6. Convert DetailActivity to WordListFragment

Hopefully you enjoyed getting the chance to migrate `DetailActivity` to `WordListFragment`. This is almost identical to migrating `MainActivity` to `LetterListFragment`. If you got stuck at any point, the steps are summarized below.

1. First, copy the companion object to `WordListFragment`.

```
companion object {  
    val LETTER = "letter"  
    val SEARCH_PREFIX = "https://www.google.com/search?q="
```

2. Then in `LetterAdapter`, in the `onClickListener()` where you perform the intent, you need to update the call to `putExtra()`, replacing `DetailActivity.LETTER` with `WordListFragment.LETTER`.

```
intent.putExtra(WordListFragment.LETTER, holder.button.text.toString())
```

3. Similarly, in `WordAdapter` you need to update the `onClickListener()` where you navigate to the search results for the word, replacing `DetailActivity.SEARCH_PREFIX` with `WordListFragment.SEARCH_PREFIX`.

```
val queryUrl: Uri = Uri.parse("${WordListFragment.SEARCH_PREFIX}${item}")
```

4. Back in `WordListFragment`, you add a binding variable of type `FragmentWordListBinding?`.

```
private var _binding: FragmentWordListBinding? = null
```

5. You then create a get-only variable so that you can reference views without having to use `?`.

```
private val binding get() = _binding!!
```

6. Then you inflate the layout, assigning the `_binding` variable and returning the root view. Remember that for fragments you do this in `onCreateView()`, not `onCreate()`.

```
override fun onCreateView(  
    inflater: LayoutInflater,  
    container: ViewGroup?,  
    savedInstanceState: Bundle?  
): View? {  
    _binding = FragmentWordListBinding.inflate(inflater, container, false)
```

```

        return binding.root
    }

```

7. Next, you implement `onViewCreated()`. This is almost identical to configuring the `recyclerView` in `onCreate()` in the `DetailActivity`. However, because fragments don't have direct access to the `intent`, you need to reference it with `activity.intent`. You have to do this in `onViewCreated()` however, as there's no guarantee the activity exists earlier in the lifecycle.

```

override fun onViewCreated(view: View, savedInstanceState: Bundle?) {
    val recyclerView = binding.recyclerView
    recyclerView.layoutManager = LinearLayoutManager(requireContext())
    recyclerView.adapter =
WordAdapter(activity?.intent?.extras?.getString(LETTER).toString(),
requireContext())

    recyclerView.addItemDecoration(
        DividerItemDecoration(context, DividerItemDecoration.VERTICAL)
    )
}

```

8. Finally, you can reset the `_binding` variable in `onDestroyView()`.

```

override fun onDestroyView() {
    super.onDestroyView()
    _binding = null
}

```

9. With all this functionality moved into `WordListFragment`, you can now delete the code from `DetailActivity`. All that should be left is the `onCreate()` method.

```

override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

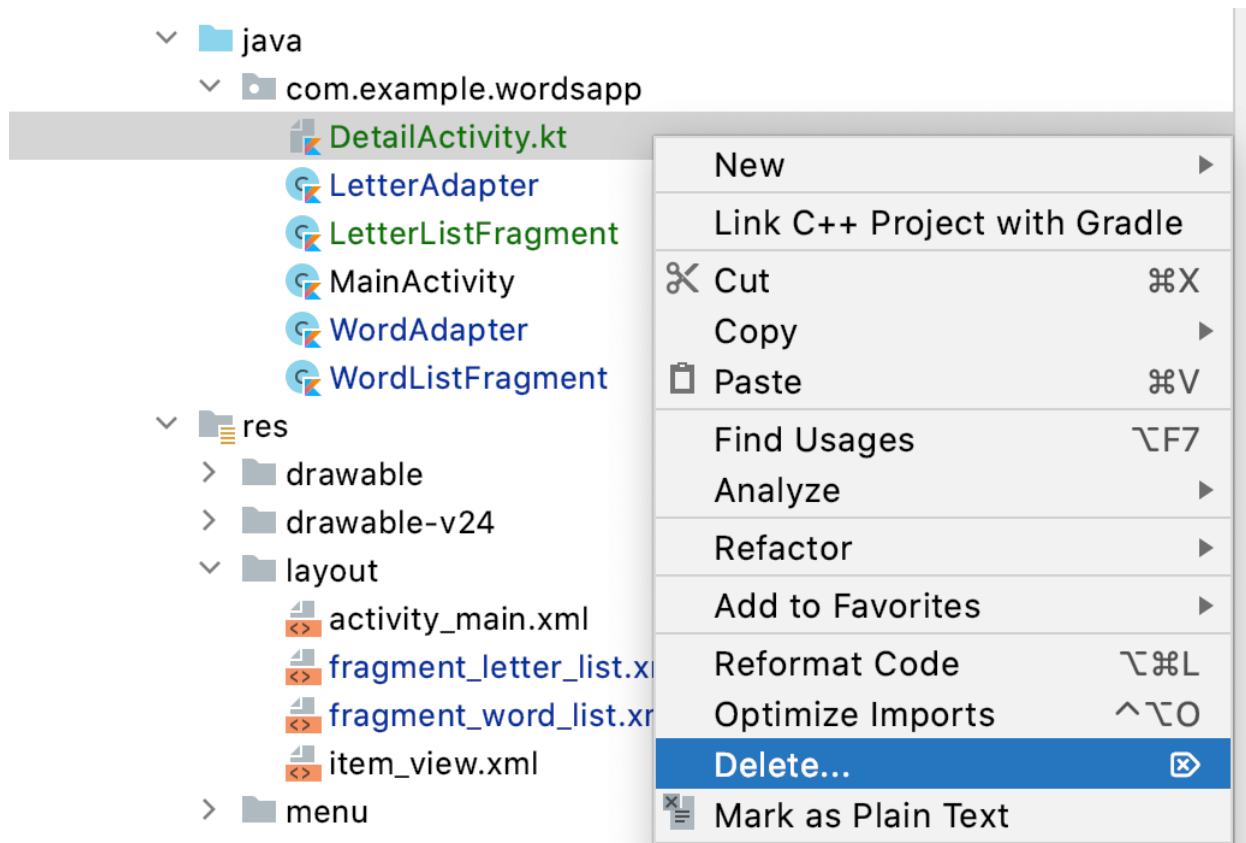
    val binding = ActivityDetailBinding.inflate(layoutInflater)
    setContentView(binding.root)
}

```

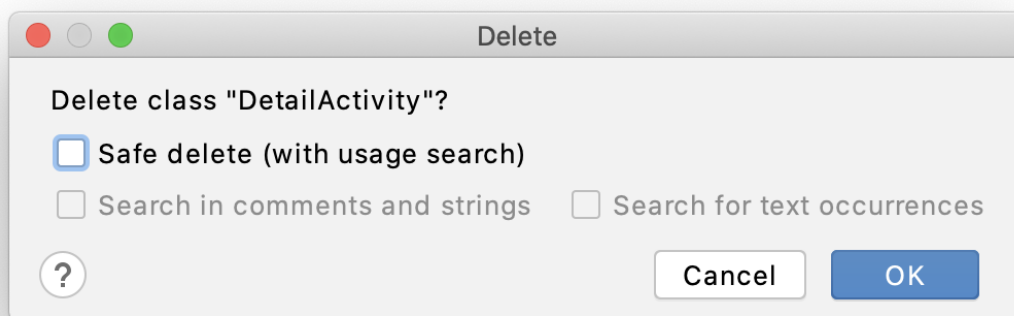
Remove DetailActivity

Now that you've successfully migrated the functionality of `DetailActivity` into `WordListFragment`, you no longer need `DetailActivity`. You can go ahead and delete both the `DetailActivity.kt` and `activity_detail.xml` as well as make a small change to the manifest.

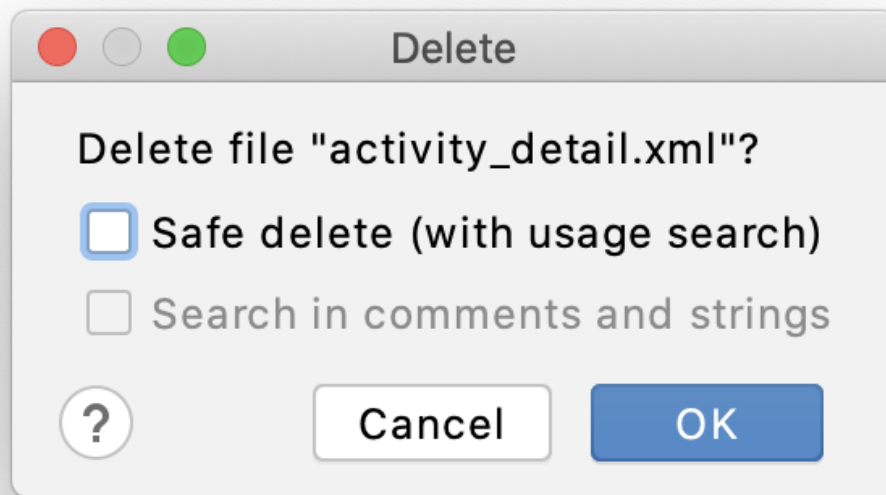
1. First, delete `DetailActivity.kt`



2. Make sure **Safe Delete** is Unchecked and click **OK**.



3. Next, delete `activity_detail.xml`. Again, make sure **Safe Delete** is unchecked.



4. Finally, as `DetailActivity` no longer exists, remove the following from `AndroidManifest.xml`.

```
<activity
    android:name=".DetailActivity"
    android:parentActivityName=".MainActivity" />
```

After deleting the detail activity, you're left with two fragments (`LetterListFragment` and `WordListFragment`) and a single activity (`MainActivity`). In the next section, you'll learn about the Jetpack Navigation component and edit `activity_main.xml` so that it can display and navigate between fragments, rather than host a static layout.

[7. Jetpack Navigation Component](#)

Android Jetpack provides the *Navigation component* to help you handle any navigation implementation, simple or complex, in your app. The Navigation component has three key parts which you'll use to implement navigation in the **Words** app.

- **Navigation Graph:** The navigation graph is an XML file that provides a visual representation of navigation in your app. The file consists of *destinations* which correspond to individual activities and fragments as well as actions between them which can be used in code to navigate from one destination to another. Just like with layout

files, Android Studio provides a visual editor to add destinations and actions to the navigation graph.

- **NavHost:** A `NavHost` is used to display destinations from a navigation graph within an activity. When you navigate between fragments, the destination shown in the `NavHost` is updated. You'll use a built-in implementation, called `NavHostFragment`, in your `MainActivity`.
- **NavController:** The `NavController` object lets you control the navigation between destinations displayed in the `NavHost`. When working with intents, you had to call `startActivity` to navigate to a new screen. With the Navigation component, you can call the `NavController`'s `navigate()` method to swap the fragment that's displayed. The `NavController` also helps you handle common tasks like responding to the system "up" button to navigate back to the previously displayed fragment.

Navigation Dependency

1. In the project-level `build.gradle` file, in **buildscript** > **ext**, below `material_version` set the `nav_version` equal to `2.5.2`.

```
buildscript {  
    ext {  
        appcompat_version = "1.5.1"  
        constraintlayout_version = "2.1.4"  
        core_ktx_version = "1.9.0"  
        kotlin_version = "1.7.10"  
        material_version = "1.7.0-alpha2"  
        nav_version = "2.5.2"  
    }  
    ...  
}
```

2. In the app-level `build.gradle` file, add the following to the dependencies group:

```
implementation "androidx.navigation:navigation-fragment-ktx:$nav_version"  
implementation "androidx.navigation:navigation-ui-ktx:$nav_version"
```

Safe Args Plugin

When you first implemented navigation in the **Words** app, you used an explicit intent between the two activities. To pass data between the two activities, you called the `putExtra()` method, passing in the selected letter.

Before you start implementing the Navigation component into the **Words** app, you'll also add something called **Safe Args**—a Gradle plugin that will assist you with type safety when passing data between fragments.

Perform the following steps to integrate `SafeArgs` into your project.

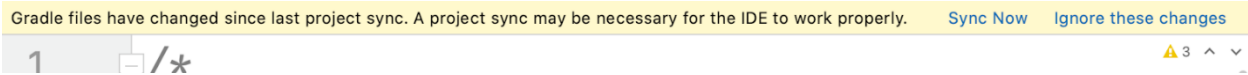
1. In the top-level `build.gradle` file, in **buildscript > dependencies**, add the following classpath.

```
classpath "androidx.navigation:navigation-safe-args-gradle-  
plugin:$nav_version"
```

2. In the app-level `build.gradle` file, within `plugins` at the top, add `androidx.navigation.safeargs.kotlin`.

```
plugins {  
    id 'com.android.application'  
    id 'kotlin-android'  
    id 'kotlin-kapt'  
    id 'androidx.navigation.safeargs.kotlin'  
}
```

3. Once you've edited the Gradle files, you may see a yellow banner at the top asking you to sync the project. Click "**Sync Now**" and wait a minute or two while Gradle updates your project's dependencies to reflect your changes.



Once syncing is complete, you're ready to move on to the next step where you'll add a navigation graph.

8. Using the Navigation Graph

Now that you have a basic familiarity with fragments and their lifecycle, it's time for things to get a bit more interesting. The next step is to incorporate the Navigation component. The *navigation component* simply refers to the collection of tools for implementing navigation, particularly between fragments. You'll be working with a new visual editor to help implement navigation between fragments; the Navigation Graph (or NavGraph for short).

What is a Navigation Graph?

The Navigation Graph (or NavGraph for short) is a virtual mapping of your app's navigation. Each screen, or fragment in your case, becomes a possible "destination" that can be navigated to. A `NavGraph` can be represented by an XML file showing how each destination relates to one another.

Behind the scenes, this actually creates a new instance of the `NavGraph` class. However, destinations from the navigation graph are displayed to the user by the `FragmentContainerView`. All you need to do is to create an XML file and define the possible destinations. Then you can use the generated code to navigate between fragments.

Use `FragmentContainerView` in `MainActivity`

Because your layouts are now contained in `fragment_letter_list.xml` and `fragment_word_list.xml`, your `activity_main.xml` file no longer needs to contain the layout for the first screen in your app. Instead, you'll repurpose `MainActivity` to contain a `FragmentContainerView` to act as the `NavHost` for your fragments. From this point forward, all the navigation in the app will take place within the `FragmentContainerView`.

1. Replace the content of the `FrameLayout` in **`activity_main.xml`** that is `androidx.recyclerview.widget.RecyclerView` with a `FragmentContainerView`. Give it an ID of `nav_host_fragment` and set its height and width to `match_parent` to fill the entire frame layout.

Replace this:

```
<androidx.recyclerview.widget.RecyclerView
    android:id="@+id/recycler_view"
    ...
    android:padding="16dp" />
```

With this:

```
<androidx.fragment.app.FragmentContainerView
    android:id="@+id/nav_host_fragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

2. Below the `id` attribute, add a `name` attribute and set it to `androidx.navigation.fragment.NavHostFragment`. While you can specify a specific fragment for this attribute, setting it to `NavHostFragment` allows your `FragmentContainerView` to navigate between fragments.

```
android:name="androidx.navigation.fragment.NavHostFragment"
```

3. Below the `layout_height` and `layout_width` attributes, add an attribute called `app:defaultNavHost` and set it equal to `"true"`. This allows the fragment container to interact with the navigation hierarchy. For example, if the system back button is pressed, then the container will navigate back to the previously shown fragment, just like what happens when a new activity is presented.

```
app:defaultNavHost="true"
```

4. Add an attribute called `app:navGraph` and set it equal to `"@navigation/nav_graph"`. This points to an XML file that defines how your app's fragments can navigate to one another. For now, the Android studio will show you an unresolved symbol error. You will address this in the next task.

```
app:navGraph="@navigation/nav_graph"
```

5. Finally, because you added two attributes with the app namespace, be sure to add the `xmlns:app` attribute to the `FrameLayout`.

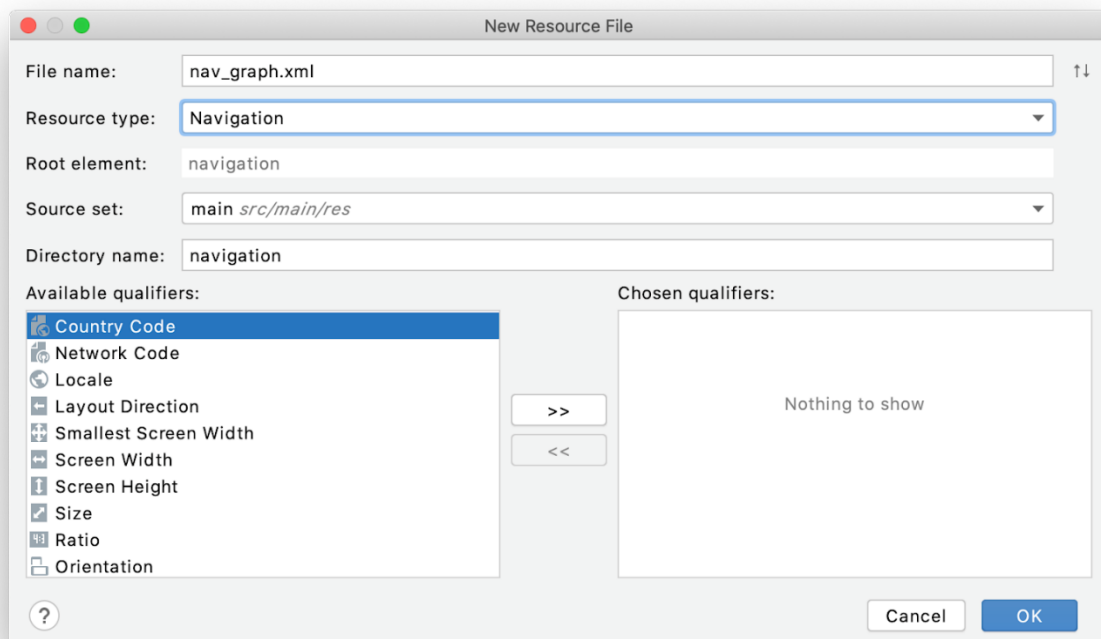
```
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:tools="http://schemas.android.com/tools"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">
```

That's all the changes in `activity_main.xml`. Next up, you'll create the `nav_graph` file.

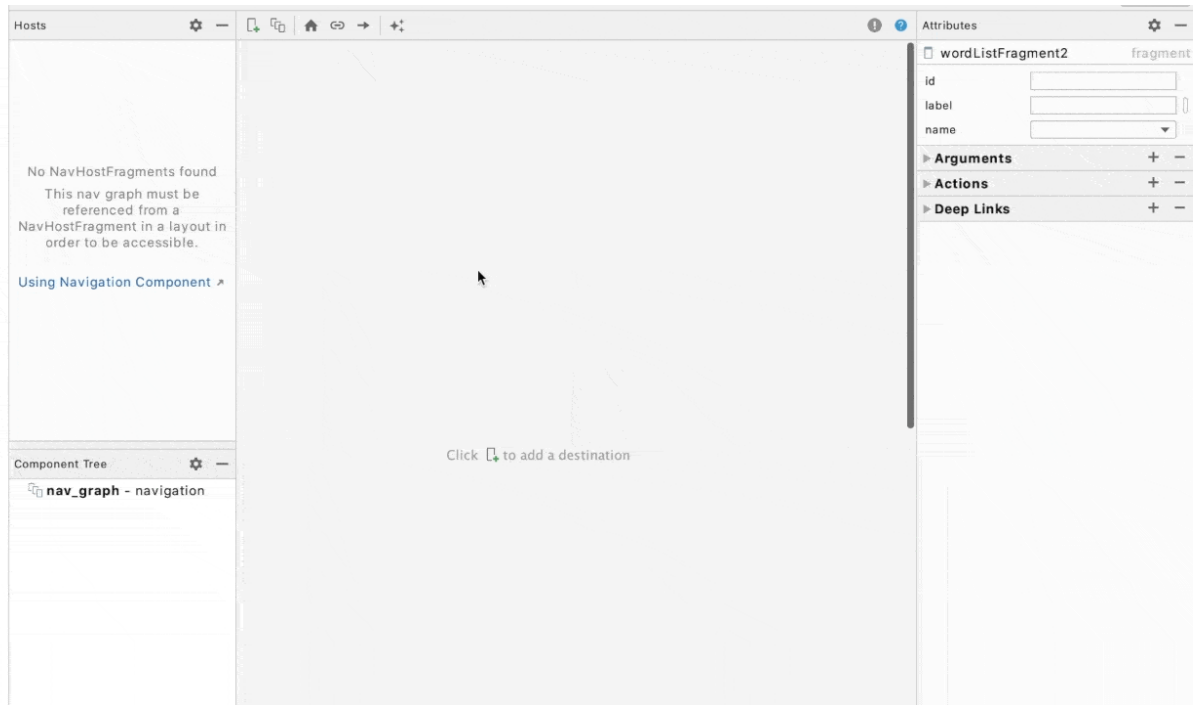
Set Up the Navigation Graph

Add a navigation graph file (**File > New > Android Resource File**) and filling the fields as follows.

- **File name:** `nav_graph.xml`. This is the same as the name you set for the `app:navGraph` attribute.
- **Resource type:** **Navigation**. The **Directory name** should then automatically change to `navigation`. A new resource folder called "navigation" will be created.



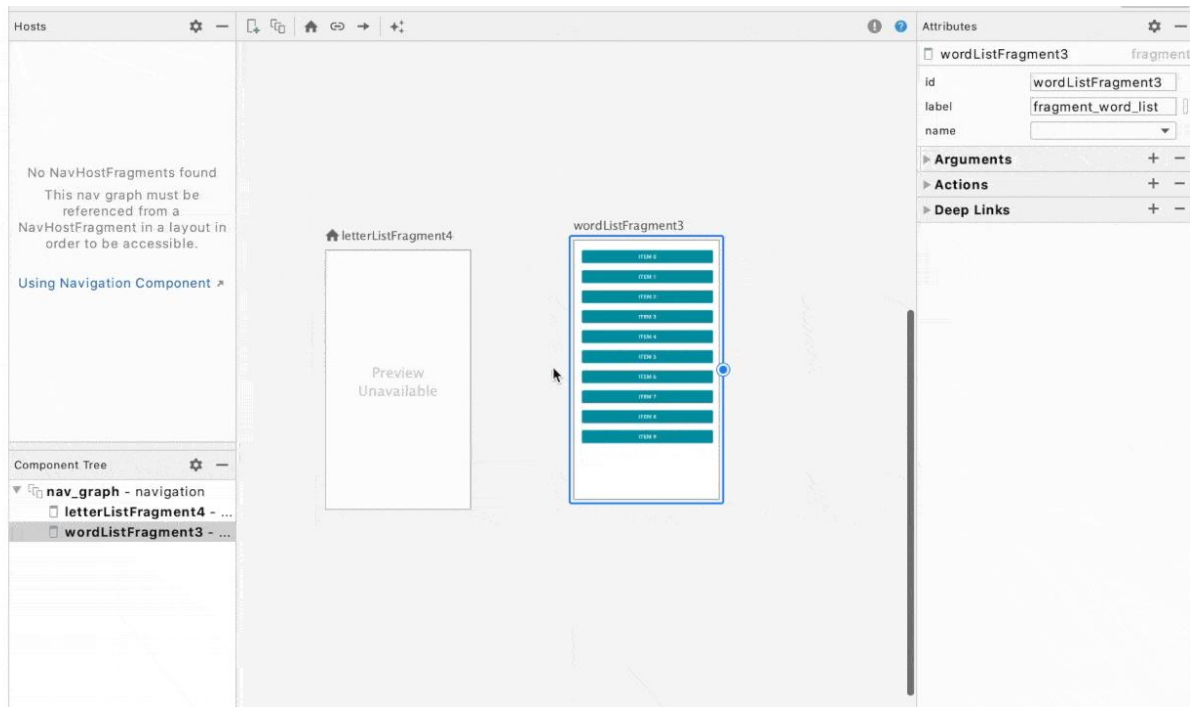
Upon creating the XML file, you're presented with a new visual editor. Because you've already referenced `nav_graph` in the `FragmentManagerView`'s `navGraph` property, to add a new destination, click the new button in the top left of the screen and create a destination for each fragment (one for `fragment_letter_list` and one for `fragment_word_list`).



Once added, these fragments should appear on the navigation graph in the middle of the screen. You can also select a specific destination using the component tree that appears on the left.

Create a navigation action

To create a navigation action between the `letterListFragment` to the `wordListFragment` destinations, hover your mouse over the `letterListFragment` destination and drag from the circle that appears on the right onto the `wordListFragment` destination.



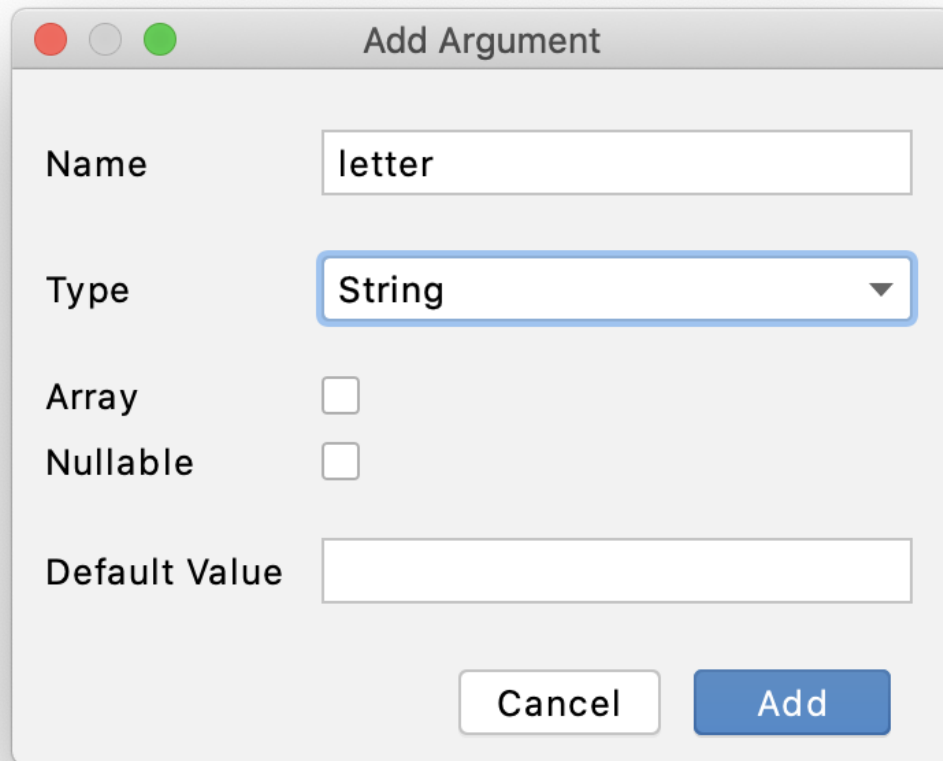
You should now see an arrow has been created to represent the action between the two destinations. Click on the arrow, and you can see in the attributes pane that this action has a name `action_letterListFragment_to_wordListFragment` that can be referenced in code.

Specify Arguments for WordListFragment

When navigating between activities using an intent, you specified an "extra" so that the selected letter could be passed to the `wordListFragment`. Navigation also supports passing parameters between destinations and plus does this in a type safe way.

Select the `wordListFragment` destination and in the attributes pane, under **Arguments**, click the plus button to create a new argument.

The argument should be called `letter` and the type should be `String`. This is where the Safe Args plugin you added earlier comes in. Specifying this argument as a string ensures that a `String` will be expected when your navigation action is performed in code.

A macOS-style dialog box titled "Add Argument". It has a title bar with red, yellow, and green window control buttons. The dialog contains five rows of controls: "Name" with a text field containing "letter"; "Type" with a dropdown menu showing "String"; "Array" with an unchecked checkbox; "Nullable" with an unchecked checkbox; and "Default Value" with an empty text field. At the bottom right are two buttons: "Cancel" and "Add".

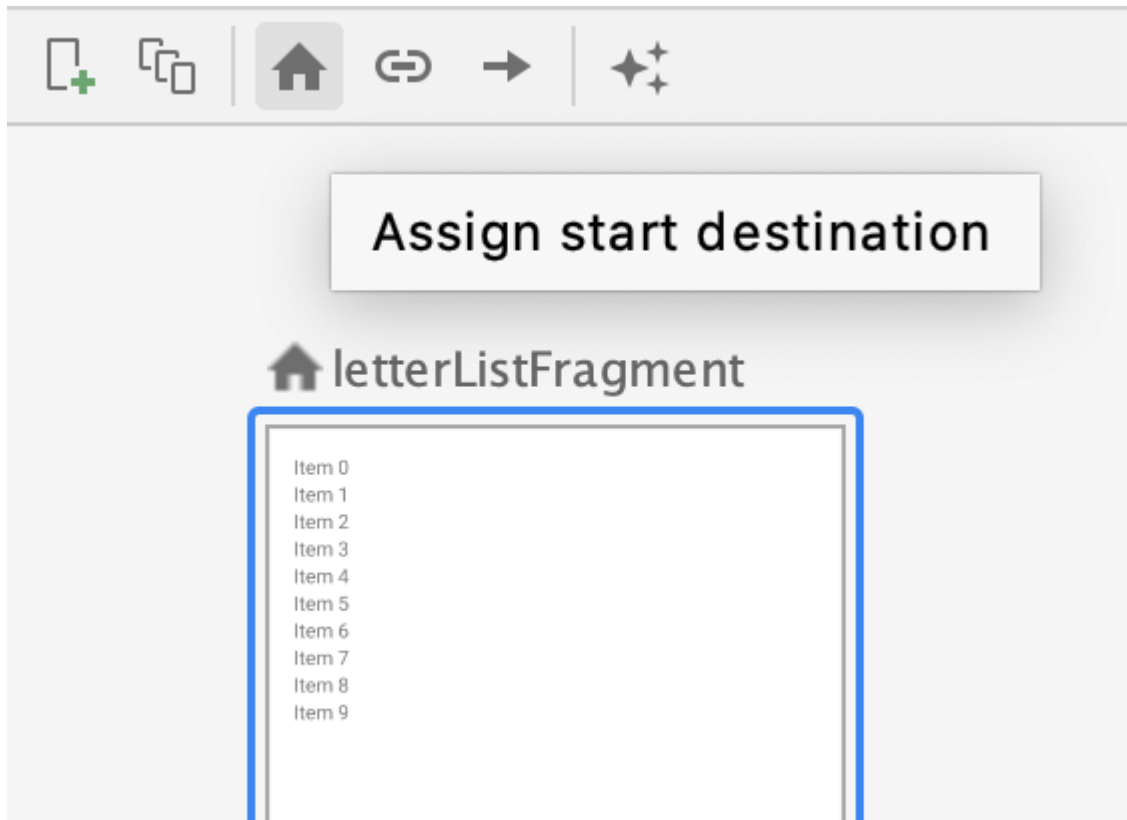
Name	letter
Type	String
Array	☐
Nullable	☐
Default Value	

Cancel Add

Setting the Start Destination

While your `NavGraph` is aware of all the needed destinations, how will the `FragmentContainerView` know which fragment to show first? On the `NavGraph`, you need to set the letter list as a start destination.

Set the start destination by selecting `letterListFragment` and clicking the **Assign start destination** button.



1. That's all you need to do with the NavGraph editor for now. At this point, go ahead and build the project. In Android Studio select **Build > Rebuild Project** from the menu bar. This will generate some code based on your navigation graph so that you can use the navigation action you just created.

Perform the Navigation Action

Open up `LetterAdapter.kt` to perform the navigation action. This only requires two steps.

1. Delete the contents of the button's `setOnClickListener()`. Instead, you need to retrieve the navigation action you just created. Add the following to the `setOnClickListener()`.

```
val action =  
LetterListFragmentDirections.actionLetterListFragmentToWordListFragment(letter  
r = holder.button.text.toString())
```

You probably don't recognize some of these class and function names and that's because they've been automatically generated after you built the project. That's where the Safe Args plugin you added in the first step comes in—the actions created on the NavGraph are turned into code that you can use. The names, however, should be fairly intuitive. `LetterListFragmentDirections` lets you refer to all possible navigation paths starting from the `letterListFragment`.

The function `actionLetterListFragmentToWordListFragment()`

is the specific action to navigate to the `wordListFragment`.

Once you have a reference to your navigation action, simply get a reference to your *NavController* (an object that lets you perform navigation actions) and call `navigate()` passing in the action.

```
holder.view.findNavController().navigate(action)
```

Configure MainActivity

The final piece of setup is in `MainActivity`. There are just a few changes needed in `MainActivity` to get everything working.

1. Create a `navController` property. This is marked as `lateinit` since it will be set in `onCreate`.

```
private lateinit var navController: NavController
```

2. Then, after the call to `setContentView()` in `onCreate()`, get a reference to the `nav_host_fragment` (this is the ID of your `FragmentContainerView`) and assign it to your `navController` property.

```
val navHostFragment = supportFragmentManager
    .findFragmentById(R.id.nav_host_fragment) as NavHostFragment
navController = navHostFragment.navController
```

3. Then in `onCreate()`, call `setupActionBarWithNavController()`, passing in `navController`. This ensures action bar (app bar) buttons, like the menu option in `LetterListFragment` are visible.

```
setupActionBarWithNavController(navController)
```

4. Finally, implement `onSupportNavigateUp()`. Along with setting `defaultNavHost` to `true` in the XML, this method allows you to handle the **up** button. However, your activity needs to provide the implementation.

```
override fun onSupportNavigateUp(): Boolean {
    return navController.navigateUp() || super.onSupportNavigateUp()
}
```

At this point, all the components are in-place to get navigation working with fragments. However, now that navigation is performed using fragments instead of the intent, the intent extra for the letter that you use in `WordListFragment` will no longer work. In the next step, you'll update `WordListFragment`, to get the `letter` argument.

NOTE: Because the `navigateUp()` function might fail, it returns a `Boolean` for whether or not it succeeds. However, you only need to call `super.onSupportNavigateUp()` if `navigateUp()`

returns `false`. This works because of the `||` operator only requires one of the conditions to be true, so if `navigateUp()` returns `true`, the right side of the `||` expression is never executed. If, however, `navigateUp()` is false, then the implementation in the parent class is called. This is called [short-circuit evaluation](#) and is a nice little programming trick to know about.

9. Getting Arguments in WordListFragment

Previously, you referenced `activity?.intent` in `WordListFragment` to access the `letter` extra. While this works, this is not a best practice, since fragments can be embedded in other layouts, and in a larger app, it's much harder to assume which activity the fragment belongs to. Furthermore, when navigation is performed using `nav_graph` and safe arguments are used, there are no intents, so trying to access intent extras is simply not going to work.

Thankfully, accessing safe arguments is pretty straightforward, and you don't have to wait until `onViewCreated()` is called either.

1. In `WordListFragment`, create a `letterId` property. You can mark this as `lateinit` so that you don't have to make it nullable.

```
private lateinit var letterId: String
```

2. Then override `onCreate()` (not `onCreateView()` or `onViewCreated()`!), add the following:

```
override fun onCreate(savedInstanceState: Bundle?) {
    super.onCreate(savedInstanceState)

    arguments?.let {
        letterId = it.getString(LETTER).toString()
    }
}
```

Because it's possible for `arguments` to be optional, notice you call `let()` and pass in a lambda. This code will execute assuming `arguments` is not null, passing in the non null arguments for the `it` parameter. If `arguments` is null, however, the lambda will not execute.

```
arguments?.let { it: Bundle
    letterId = it.getString(LETTER).toString()
}
```

While not part of the actual code, Android Studio provides a helpful hint to make you aware of the `it` parameter.

What exactly is a `Bundle`? Think of it as a key-value pair used to pass data between classes, such as activities and fragments. Actually, you've already used a bundle when you called `intent?.extras?.getString()` when performing an intent in the first version of this app. Getting the string from arguments when working with fragments works exactly the same way.

3. Finally, you can access the `letterId` when you set the recycler view's adapter. Replace `activity?.intent?.extras?.getString(LETTER).toString()` in `onViewCreated()` with `letterId`.

```
recyclerView.adapter = WordAdapter(letterId, requireContext())
```

You did it! Take a moment to run your app. It's now able to navigate between two screens, without any intents, and all in a single activity.

10. Update Fragment Labels

You've successfully converted both screens to use fragments. Before any changes were made, the app bar for each fragment had a descriptive title for each activity contained in the app bar. However, after converting to use fragments, this title is missing from the detail activity.



Words

AARGH

ABOUT

ACRID

ANECDOTE

AWESOME

Fragments have a property called "label" where you can set the title which the parent activity will know to use in the app bar.

1. In `strings.xml`, after the app name, add the following constant.

```
<string name="word_list_fragment_label">Words That Start With  
{letter}</string>
```

2. You can set the label for each fragment on the navigation graph. Go back into `nav_graph.xml` and select `letterListFragment` in the component tree, and in the attributes pane, set the label to the `app_name` string:

letterListFragment		fragment
id	letterListFragment	
label	@string/app_name	
name		
► Arguments		+ -

3. Select `wordListFragment` and set the label to `word_list_fragment_label`:

wordListFragment		fragment
id	wordListFragment	
label	@string/word_list_fragment_label	
name		
▼ Arguments		+ -

Congratulations on making it this far! Run your app one more time and you should see everything just as it was at the start of the codelab, only now, all your navigation is hosted in a single activity with a separate fragment for each screen.

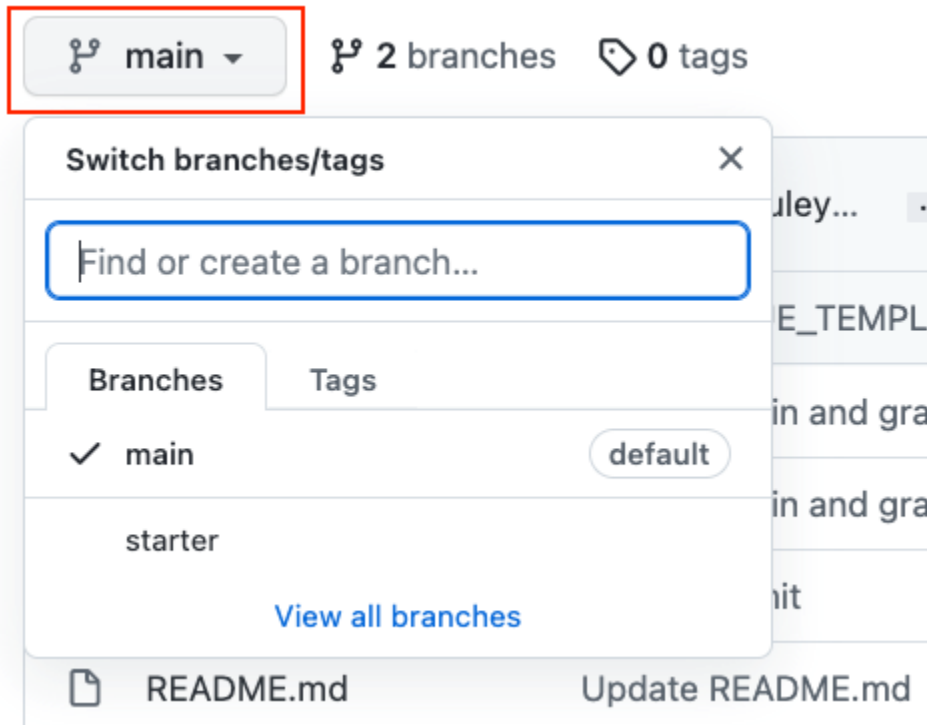
11. Solution code

The solution code for this codelab is in the project shown below.

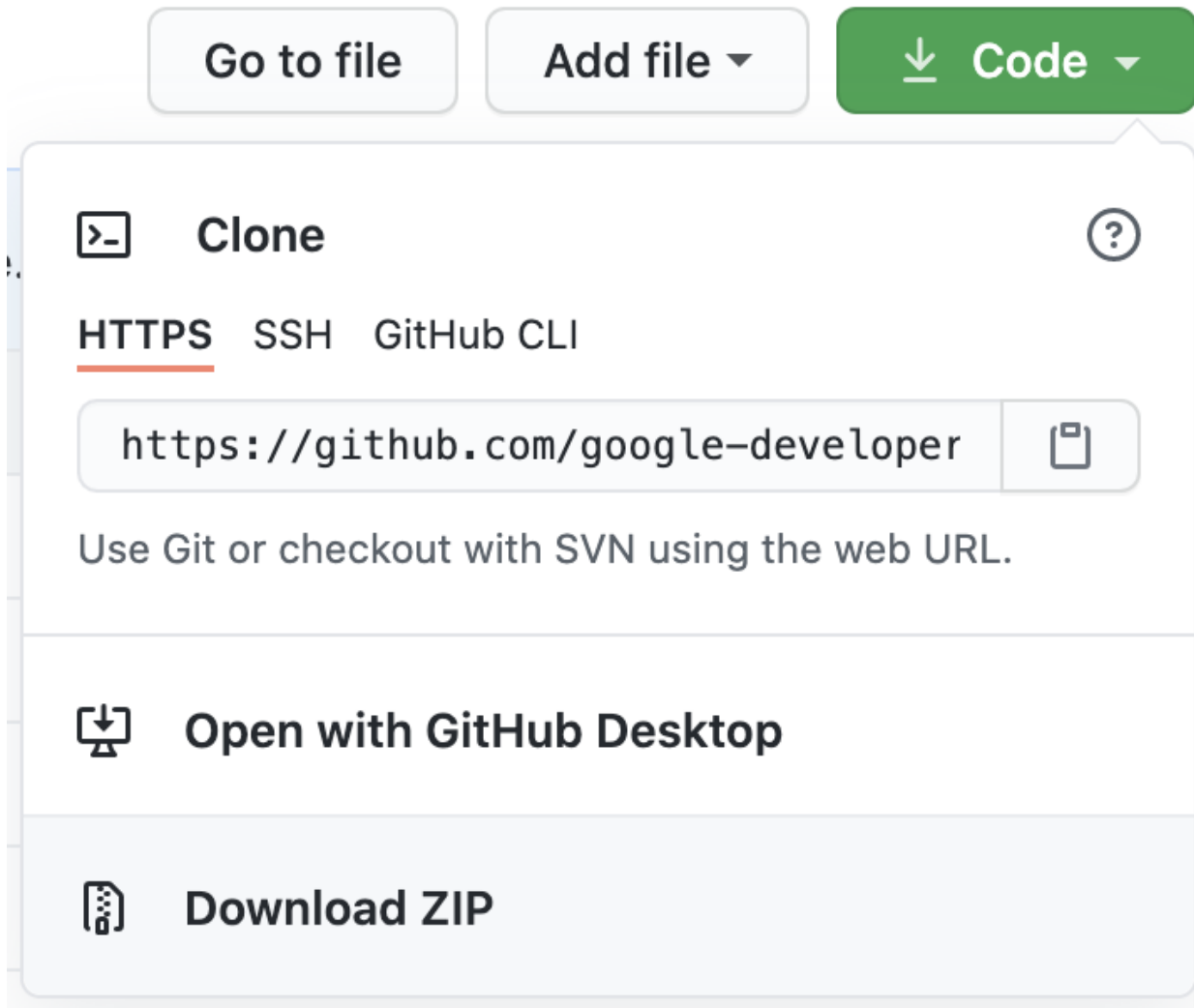
Solution Code URL: <https://github.com/google-developer-training/android-basics-kotlin-words-app>

Branch: main

1. Navigate to the provided GitHub repository page for the project.
2. Verify that the branch name matches the branch name specified in the codelab. For example, in the following screenshot the branch name is **main**.



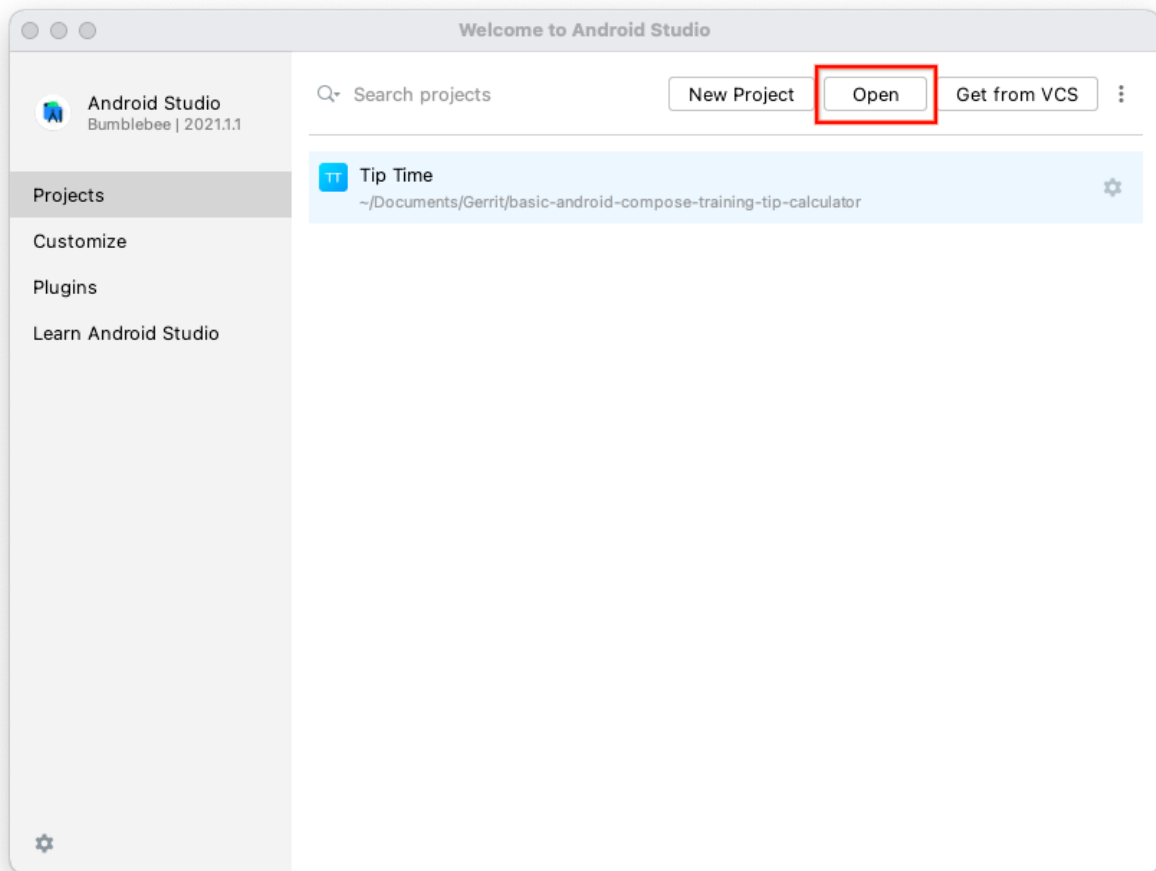
3. On the GitHub page for the project, click the **Code** button, which brings up a popup.



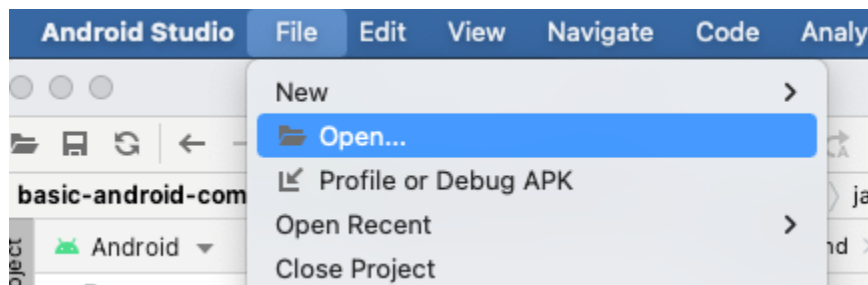
4. In the popup, click the **Download ZIP** button to save the project to your computer. Wait for the download to complete.
5. Locate the file on your computer (likely in the **Downloads** folder).
6. Double-click the ZIP file to unpack it. This creates a new folder that contains the project files.

Open the project in Android Studio

1. Start Android Studio.
2. In the **Welcome to Android Studio** window, click **Open**.



Note: If Android Studio is already open, instead, select the **File > Open** menu option.



3. In the file browser, navigate to where the unzipped project folder is located (likely in your **Downloads** folder).
4. Double-click on that project folder.
5. Wait for Android Studio to open the project.



6. Click the **Run** button to build and run the app. Make sure it builds as expected.

12. Summary

- Fragments are reusable pieces of UI that can be embedded in activities.
- The lifecycle of a fragment differs from the lifecycle of an activity, with view setup occurring in `onViewCreated()`, rather than `onCreateView()`.
- A `FragmentManager` is used to embed fragments in other activities and can manage navigation between fragments.

Use the Navigation Component

- Setting the `navGraph` attribute of a `FragmentManager` allows you to navigate between fragments within an activity.
- The `NavGraph` editor allows you to add navigation actions and specify arguments between different destinations.
- While navigating using intents requires you to pass in extras, the Navigation component uses `SafeArgs` to auto-generate classes and methods for your navigation actions, ensuring type safety with arguments.

Use cases for fragments

- Using the Navigation component, many apps can manage their entire layout within a single activity, with all navigation occurring between fragments.
- Fragments make common layout patterns possible, such as master-detail layouts on tablets, or multiple tabs within the same activity.

13. Learn more

- [Fragments](#)
- [Fragment Class Reference](#)
- [SafeArgs](#)
- [Bundle Class Reference](#)
- [Null Safety in Kotlin](#)
- [FragmentManager](#)